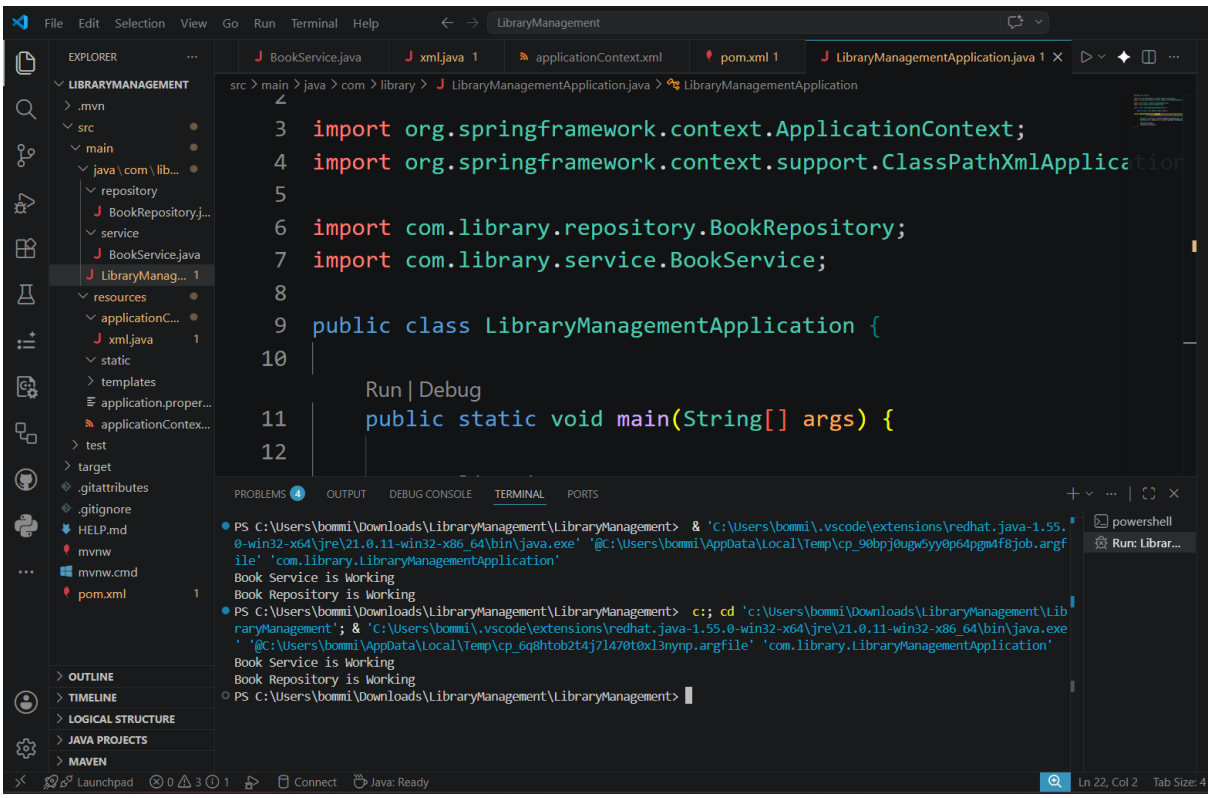
Exercise 1: Configuring a Basic Spring Application



[BookRepository.java]

```
package com.library.repository;

public class BookRepository {

    public void display() {

        System.out.println("Book Repository is Working");

    }

}
```

[BookService.java]

```
package com.library.service;


public class BookService {
```

```
    public void display() {
        System.out.println("Book Service is Working");
    }
}
```

applicationContext.xml

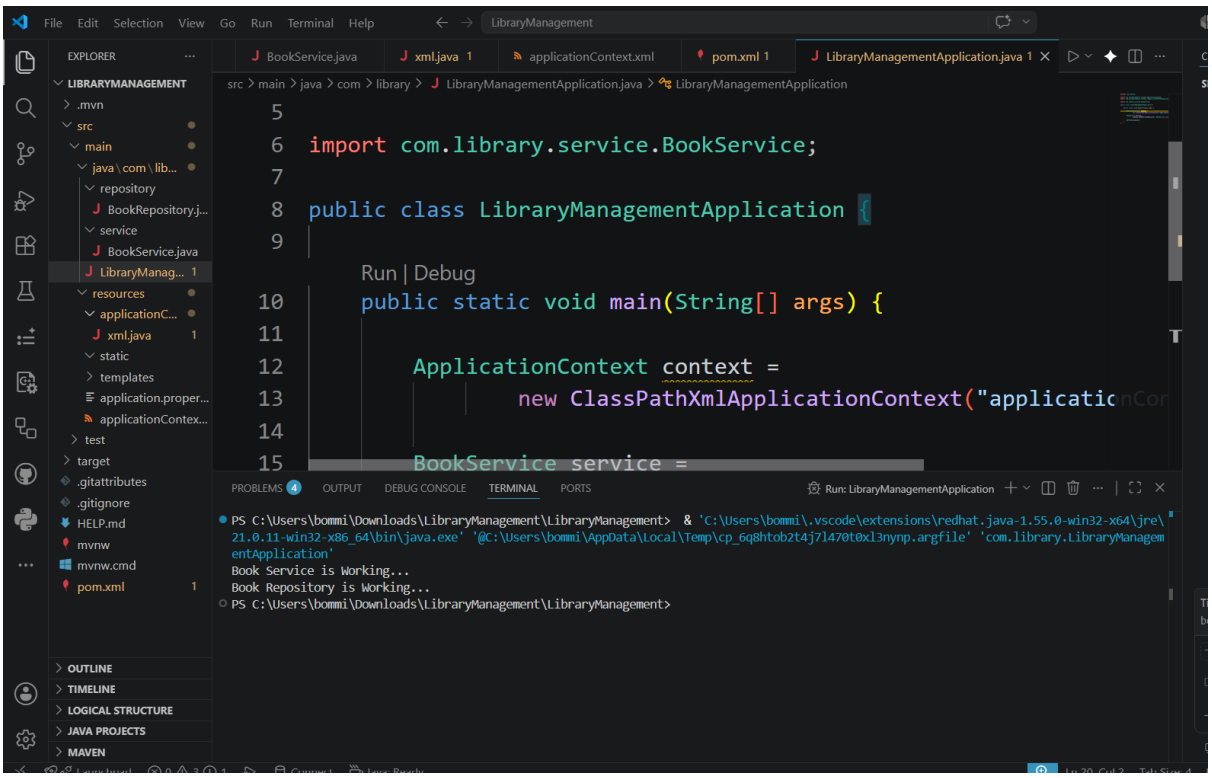
```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService"/>

</beans>
```

LibraryManagementApplication.java

```
package com.library;

import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.repository.BookRepository;
import com.library.service.BookService;

public class LibraryManagementApplication {

   public static void main(String[] args) {

      ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

      BookService service = context.getBean("bookService", BookService.class);
      BookRepository repository = context.getBean("bookRepository",
BookRepository.class);

      service.display();
      repository.display();
   }
}
```

Exercise 2: Implementing Dependency Injection



BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    // Setter Injection

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

    }

    public void display() {

        System.out.println("Book Service is Working...");
```

```
        bookRepository.display();
    }
}
```

BookRepository.java

```
package com.library.repository;
public class BookRepository {
    public void display() {
        System.out.println("Book Repository is Working...");
    }
}
```

applicationContext.xml

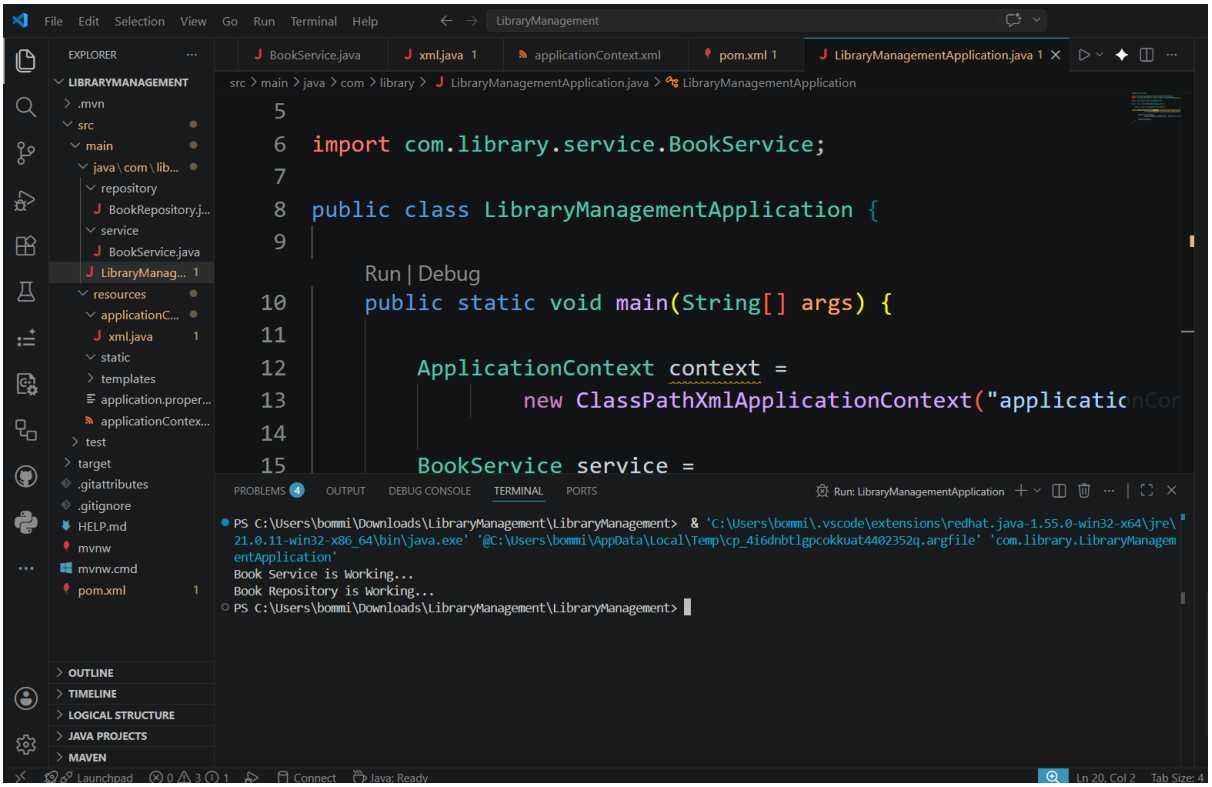
```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">
    <!-- Repository Bean -->
    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <!-- Service Bean -->
    <bean id="bookService"
          class="com.library.service.BookService">
```

```
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
</beans>
```

LibraryManagementApplication.java

```
package com.library;


import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;


import com.library.service.BookService;


public class LibraryManagementApplication {


    public static void main(String[] args) {


        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");


        BookService service =
                context.getBean("bookService", BookService.class);


        service.display();
    }
}
```

Exercise 4: Creating and Configuring a Maven Project



[pom.xml](#)

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>

    <artifactId>LibraryManagement</artifactId>

    <version>1.0</version>

    <properties>
```

```
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <dependencies>
    <!-- Spring Context -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
    </dependency>
    <!-- Spring AOP -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.30</version>
    </dependency>
    <!-- AspectJ Weaver -->
    <dependency>
      <groupId>org.aspectj</groupId>
      <artifactId>aspectjweaver</artifactId>
      <version>1.9.22</version>
    </dependency>
    <!-- Spring Web MVC -->
    <dependency>
      <groupId>org.springframework</groupId>
```

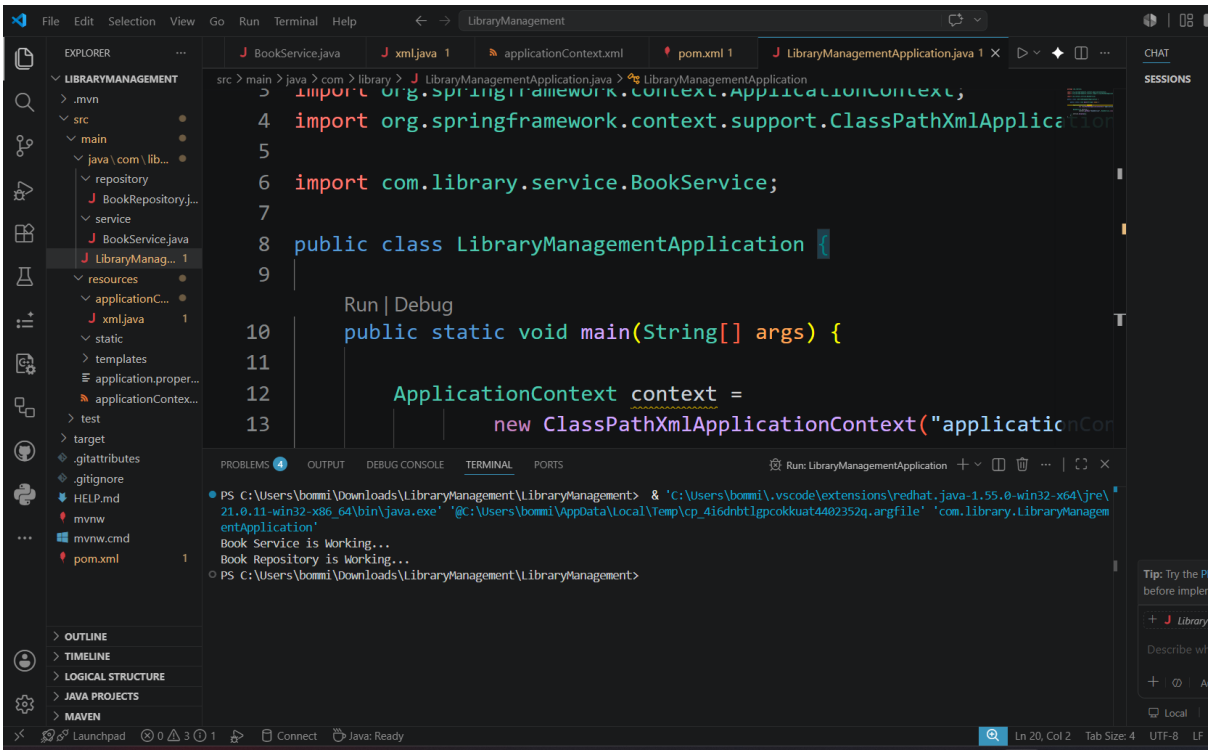
```
        <artifactId>spring-webmvc</artifactId>
        <version>5.3.30</version>
      </dependency>
    </dependencies>
    <build>
      <plugins>
        <!-- Maven Compiler Plugin -->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.11.0</version>
          <configuration>
            <source>1.8</source>
            <target>1.8</target>
          </configuration>
        </plugin>
      </plugins>
    </build>
</project>
```

LibraryManagementApplication.java

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;


import com.library.service.BookService;
```

```
public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =

                new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =

                context.getBean("bookService", BookService.class);

        service.display();

    }

}
```

Exercise 5: Configuring the Spring IoC Container



BookRepository.java

```
package com.library.repository;

public class BookRepository {

   public void display() {

      System.out.println("Book Repository is Working...");

   }

}
```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

   private BookRepository bookRepository;

   // Setter Injection

   public void setBookRepository(BookRepository bookRepository) {

      this.bookRepository = bookRepository;

   }

   public void display() {

      System.out.println("Book Service is Working...");

      bookRepository.display();

   }

}
```

applicationContext.xml

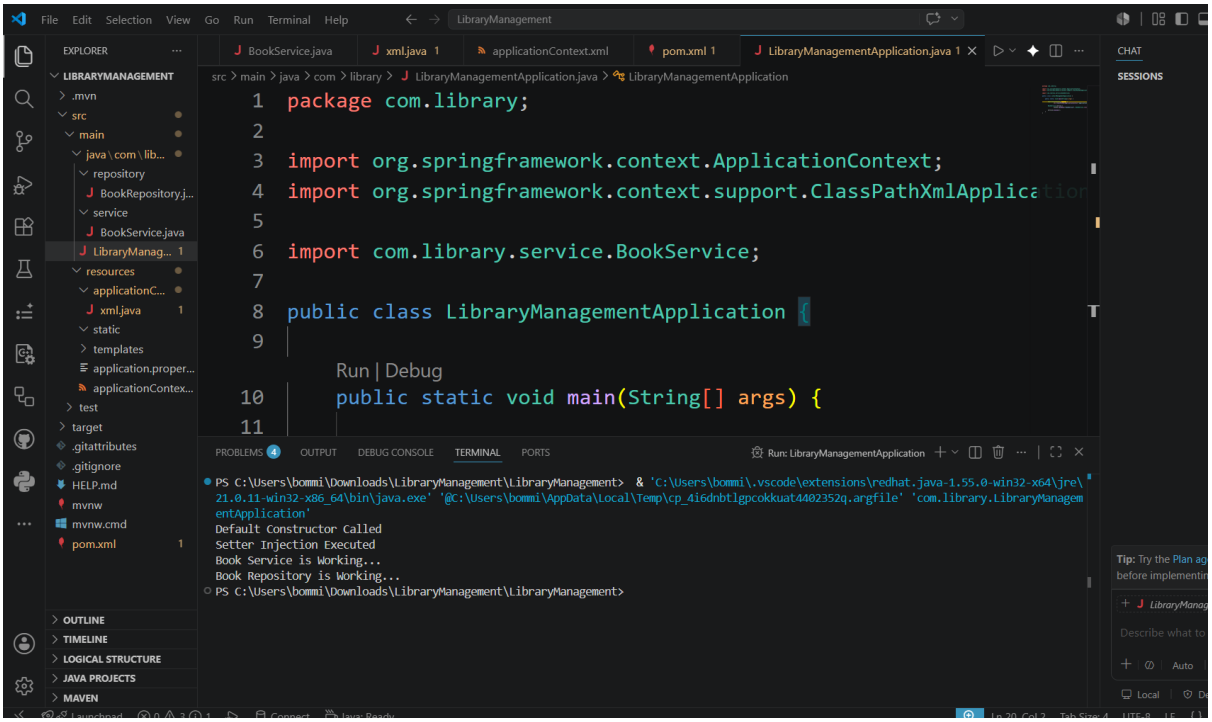
```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"

      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <!-- Repository Bean -->
  <bean id="bookRepository"
      class="com.library.repository.BookRepository"/>
  <!-- Service Bean -->
  <bean id="bookService"
      class="com.library.service.BookService">
    <property name="bookRepository" ref="bookRepository"/>
  </bean>
</beans>
```

LibraryManagementApplication.java

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
  public static void main(String[] args) {
    ApplicationContext context =
        new ClassPathXmlApplicationContext("applicationContext.xml");
    BookService service =
        context.getBean("bookService", BookService.class);
    service.display();
 } }
```

Exercise 7: Implementing Constructor and Setter Injection



BookRepository.java

```
package com.library.repository;

public class BookRepository {

    public void display() {

        System.out.println("Book Repository is Working...");

    }

}
```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;
```

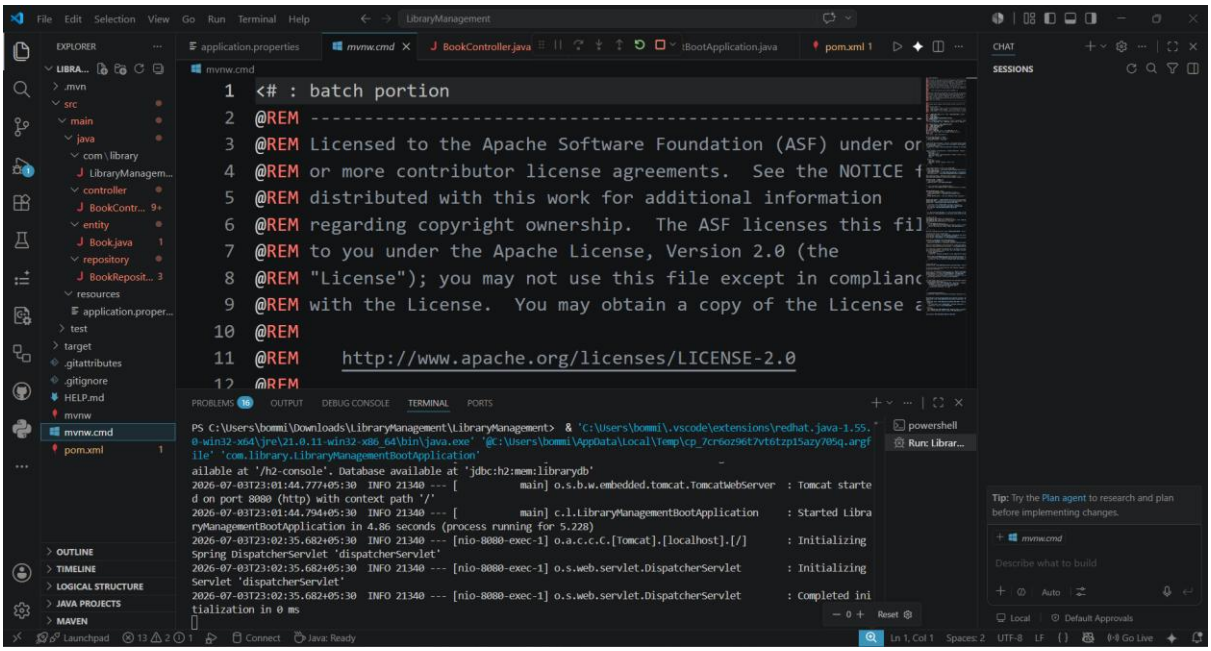
```
    // Default Constructor
    public BookService() {
        System.out.println("Default Constructor Called");
    }
    // Constructor Injection
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Constructor Injection Executed");
    }
    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Setter Injection Executed");
    }
    public void display() {
        System.out.println("Book Service is Working...");
        bookRepository.display();
    }
}
```

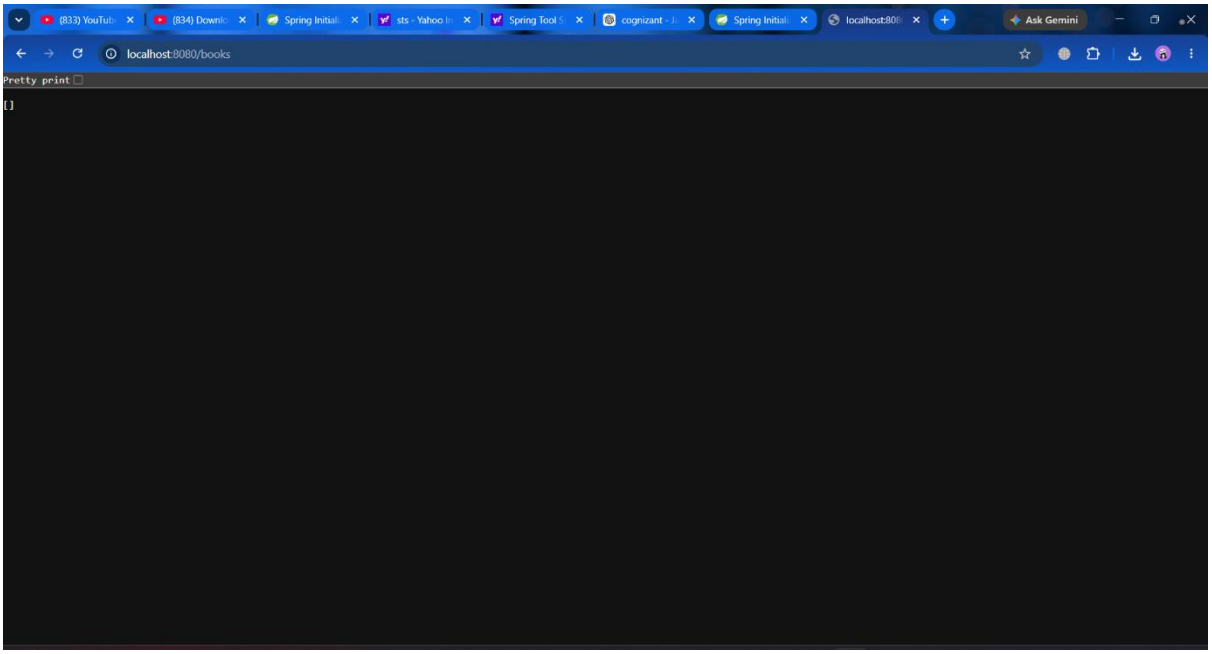
applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
```

```
    https://www.springframework.org/schema/beans/spring-beans.xsd">

  <!-- Repository Bean -->

  <bean id="bookRepository"

      class="com.library.repository.BookRepository"/>


  <!-- Service Bean -->

  <bean id="bookService"

      class="com.library.service.BookService">

    <!-- Setter Injection -->

    <property name="bookRepository"

          ref="bookRepository"/>

  </bean>

</beans>
```

Exercise 9: Creating a Spring Boot Application





get [] as the output. database has no books